

Coco App - Analisis Tech Stack

Coco App — Análisis profundo del tech stack

Producto · Almacenamiento · Mantenibilidad · Escalabilidad

Fecha: 6 de agosto de 2026 · Autor: Andres (Vettx) · Para: decisión de arquitectura de *Coco App*

1. Resumen ejecutivo (el veredicto primero)

La conclusión del análisis es que **el stack ya definido es el correcto** para el escenario real de Coco App, y no por inercia: es la mejor combinación de valor cuando el producto tiene dos vidas — una **etapa personal de hasta ~50 usuarios sobre Hostinger a costo cero**, y una eventual **etapa vendible que se migra a infraestructura profesional**. El stack recomendado es:

- **Frontend:** React 19 + Vite + TypeScript (SPA estática).
- **Backend:** Node.js (LTS) + NestJS + TypeScript.
- **ORM:** Prisma.
- **Base de datos:** MariaDB en Hostinger (etapa 1) → base gestionada (etapa 2).
- **Auth:** Firebase Authentication.
- **Procesamiento de documentos (OCR):** en el cliente (Tesseract.js / pdf.js), detrás de una interfaz *intercambiable*.

La tesis que sostiene todo el análisis es **“portable-first”**: elegir tecnologías estándar y desacopladas que **corran gratis hoy en Hostinger** y que, el día que Coco valga la pena venderse, se **muevan a infra escalable sin reescribir** — solo relocalizando piezas. La decisión más importante no es *qué* tecnología, sino *no quedar amarrado* a ninguna.

Sobre las dos preguntas que dispararon este análisis: **sí, Hostinger corre MariaDB (no MySQL), y sí, es una opción perfectamente buena** — de hecho es la correcta para la etapa 1. Y **sí, la arquitectura aguanta 50 usuarios en Hostinger con holgura**, principalmente porque el trabajo pesado (OCR) ocurre en el navegador y casi no toca el servidor.

2. Contexto y criterios de evaluación

2.1 Perfil del producto

Coco App es una aplicación de finanzas personales que registra movimientos, gastos fijos, presupuestos, deudas y metas, y que **importa información desde screenshots y extractos mediante OCR en el navegador**. Tres rasgos de este producto condicionan toda decisión técnica:

1. **Carga de servidor intrínsecamente baja.** El uso típico son lecturas (dashboard, listados) y escrituras esporádicas (registrar un movimiento, confirmar una importación). Lo único costoso —el OCR y el parsing— corre en el dispositivo del usuario, **no** en el servidor. Esto cambia radicalmente el dimensionamiento: el backend es una API JSON liviana, no un procesador de imágenes.
2. **Datos sensibles pero pequeños.** Son datos financieros (máxima confidencialidad) pero de bajo volumen: montos, fechas, categorías, texto. Nada de blobs pesados en el servidor (las imágenes viven en el navegador).
3. **Multiusuario latente.** Nace para un dueño, pero con el modelo de datos scopeado por `user_id` desde el día uno; volverse multiusuario es *quitar una suposición*, no rediseñar.

2.2 Las dos etapas (lo que el usuario definió)

- **Etapla 1 — Personal + piloto (hoy):** uso propio, con capacidad de **soportar al menos ~50 usuarios**, sobre **Hostinger** (Business/Cloud ya pagado) y **sin costo adicional**.
- **Etapla 2 — Producto vendible (condicional):** si Coco funciona y se decide comercializar, se **migra a infraestructura profesional** (base gestionada, hosting de aplicaciones escalable, observabilidad, CI/CD).

Esto es liberador para el diseño: **no hay que construir hoy para millones**. Hay que construir para 50 usuarios a costo cero, pero con decisiones que *no cierren* el camino a la etapa 2.

2.3 Ejes de evaluación

Cada capa del stack se evalúa contra cinco ejes:

Eje	Pregunta que responde
Adecuación al producto	¿Resuelve bien lo que Coco necesita?
Almacenamiento / dónde viven los datos	¿Es apropiado y seguro para datos financieros, gratis en Hostinger?
Mantenibilidad	¿Puede el dueño mantenerlo solo, con bajo esfuerzo y baja carga cognitiva?
Escalabilidad / portabilidad	¿Aguanta 50 usuarios hoy y se migra sin reescritura mañana?
Costo	¿Respeta el “cero costo recurrente” en la etapa 1?

3. La tesis: arquitectura “portable-first”

El error clásico al pensar en escalabilidad es sobre-invertir temprano: montar Kubernetes, colas y microservicios para un producto que tiene un usuario. El error opuesto es amarrarse a una plataforma propietaria barata que luego es imposible de dejar. La estrategia correcta para Coco está en el medio: **tecnologías estándar, desacopladas y portables**, desplegadas hoy de la forma más simple y gratuita posible.

El principio operativo: **cada pieza del stack debe poder “levantarse y moverse” (lift-and-shift) a otra infraestructura sin reescribir la lógica de negocio**. Lo que hace esto posible o imposible:

- **Amarra (lock-in) y hay que evitar:** funciones propietarias de una plataforma (p. ej. lógica en Firebase Cloud Functions, tipos exclusivos de un motor de base de datos, un framework atado a un runtime de un solo proveedor).
- **No amarra (portable) y hay que preferir:** Node.js estándar, SQL estándar vía un ORM que abstraee el motor, un frontend estático que se sirve desde cualquier CDN, un proveedor de identidad cuyos usuarios se pueden exportar.

Todo el stack de Coco cae del lado portable. Por eso la migración de la etapa 1 a la etapa 2 es un **traslado**, no una **reconstrucción**. Ese es el corazón del análisis.

4. Dimensionamiento: ¿aguenta Hostinger 50 usuarios?

Respuesta corta: **sí, con holgura**, y el margen es grande gracias al OCR en el cliente.

4.1 Recursos reales de Hostinger

Según los límites publicados (marzo 2026): el plan **Business** entrega **~1.536 MB de RAM, 2 núcleos CPU burstables, ~60 procesos y 30 entry processes**; el plan **Cloud Startup** sube a **3 GB de RAM y núcleos dedicados** (sin la cuota de CPU burstable) con 80 entry processes ([Hostinger resource limits, 2026](#)). La Node.js Web App corre como **proceso persistente** en ese entorno.

4.2 La carga real de 50 usuarios de una app de finanzas

Hagamos el cálculo servilleta. Un usuario de finanzas personales interactúa pocas veces al día, mayormente leyendo. Supongamos un uso activo generoso de ~200 peticiones API por usuario por día:

- 50 usuarios × 200 req/día ≈ **10.000 req/día** ≈ **0,12 req/s** en promedio.
- Con picos concentrados, quizás **2-5 req/s** en el peor momento.

Un proceso NestJS despacha peticiones JSON simples (consultas Prisma indexadas y scopeadas por `user_id`) en pocos milisegundos. **2-5 req/s es carga trivial**; un solo proceso Node con 1,5 GB de RAM la atiende sin sudar. El consumo de RAM de una app NestJS ronda 100-250 MB en reposo; sobra memoria.

El factor decisivo: lo caro de Coco —OCR y parsing de imágenes/PDF— **no ocurre en el servidor**. Corre en el navegador de cada usuario. Por eso Coco escala en usuarios sin cargar el backend: 50 personas haciendo OCR local es 50 CPUs de clientes trabajando en paralelo, no la CPU de Hostinger. Este es, probablemente, el rasgo arquitectónico que más habilita la etapa 1 a costo cero.

4.3 Base de datos y conexiones

- **Volumen:** 50 usuarios × unos miles de movimientos cada uno, acumulados por años, dan del orden de cientos de miles de filas — **decenas de MB**, muy por debajo del disco (100 GB) y de lo que MariaDB maneja con índices por `user_id`.

- **Conexiones:** el punto de atención real del hosting compartido es el límite de conexiones concurrentes de MariaDB (`max_user_connections`, típicamente bajo en compartido). Con **un solo proceso Node** y el **pool de Prisma configurado bajo** (`connection_limit=5-10` en la `DATABASE_URL`), se respeta ese techo sin problema para 50 usuarios. Es una regla de configuración, no un rediseño.

4.4 Los techos honestos (cuándo Business se queda corto)

Business aguanta 50 usuarios cómodo. Los límites aparecen **más allá** de la etapa 1, no en ella:

- La **CPU burstable** tiene una cuota horaria no publicada; si el uso se volviera sostenido y pesado, Hostinger *throttlea*. Para tráfico liviano de API no es un problema; para cientos de usuarios activos sí sería una señal de migrar.
- **Un solo proceso, sin autoescalado:** si el proceso cae, hay downtime hasta el reinicio. Aceptable para uso personal; inaceptable para un producto con clientes pagando.
- **Respaldos manuales** (`cron + mysqldump`): suficiente para etapa 1, insuficiente para SLA de producto.

Recomendación de dimensionamiento: Business cubre 50 usuarios; si quieres margen tranquilo (más RAM, CPU dedicada sin cuota), **Cloud Startup** es el escalón natural dentro de Hostinger y sigue siendo económico. Ninguno de los dos exige salir de “cero costo adicional” si ya tienes el plan.

5. Análisis capa por capa

Cada capa se juzga por su adecuación, mantenibilidad, escalabilidad/portabilidad y su “movimiento al escalar”.

5.1 Almacenamiento — MariaDB en Hostinger

Qué es y por qué está bien. Hostinger corre **MariaDB**, un fork de MySQL creado por el autor original de MySQL (Monty Widenius) cuando Oracle lo adquirió. No es una rareza: MariaDB es el motor por defecto en la mayoría de distribuciones Linux y de paneles de hosting, y potencia una porción enorme de la web (incluida gran parte de WordPress del mundo). Es maduro, estable y, para el 99% de los usos, **funcionalmente equivalente a MySQL**. Que “nunca lo hubieras oído” es normal: viaja escondido detrás de la etiqueta genérica “MySQL/base de datos” de los paneles.

Adecuación al producto. Coco usa tipos y operaciones estándar (tablas relacionales, `DECIMAL` para dinero, `ENUM`, `DATE / DATETIME`, índices, claves foráneas, transacciones ACID con InnoDB). Todo eso es pan de cada día para MariaDB. No hay nada en Coco que MariaDB no haga bien.

Mantenibilidad. Se administra con phpMyAdmin desde hPanel y con cualquier cliente MySQL/MariaDB. Prisma se encarga del esquema y las migraciones, así que el mantenimiento del día a día es mínimo.

Portabilidad / escalabilidad. Aquí está la clave que despeja la preocupación: **Prisma trata a MariaDB con el mismo provider = “mysql”**, así que el código no “sabe” ni le importa si detrás hay MySQL o MariaDB. El día de la etapa 2, mover los datos a un **MySQL/MariaDB gestionado** (o incluso a **PostgreSQL** con algo de trabajo de migración que Prisma facilita) es un cambio de `DATABASE_URL` y una migración de datos, no una reescritura de la aplicación. El único matiz técnico documentado: usar el **conector estándar de Prisma** (no el driver adapter opcional `@prisma/adapters-mariadb`, que tiene un bug conocido con columnas `JSON`), y mantener el esquema en **SQL portable** (que ya es el caso).

¿Es MariaDB “la mejor” opción? Para la etapa 1 —gratis, incluida, madura, compatible con Prisma— **sí, es la mejor relación valor/riesgo**. Para un producto a gran escala, muchos equipos prefieren **PostgreSQL** por su riqueza de tipos y ecosistema; ese sería el destino natural en la etapa 2 si el producto crece. Pero adoptar Postgres hoy significaría **salir de Hostinger y sumar costo/complejidad** sin necesidad. La decisión correcta es: **MariaDB ahora, Postgres/MySQL gestionado cuando (y si) se venda**. Prisma hace ese salto viable.

Movimiento al escalar: MariaDB (Hostinger) → MySQL/Postgres gestionado (Neon, un MySQL gestionado, o un Postgres en VPS). Solo cambia `DATABASE_URL` + migración de datos.

5.2 Backend — Node.js + NestJS

Adecuación. NestJS estructura la API en módulos, controladores, servicios y DTOs — un molde que calza exactamente con los 10 módulos de Coco (movimientos, presupuestos, deudas, etc.). Trae de fábrica lo que una app financiera necesita: validación, guards de autenticación globales, inyección de dependencias, filtros de error e interceptores.

Mantenibilidad (fuerte). Es **el mismo stack que Vettx**, donde ya tienes experticia y skills. Eso baja la carga cognitiva a casi cero: mantienes Coco con los mismos patrones que ya usas a diario. Su estructura opinada también significa que

un tercero (o el “tú” del futuro) navega el código sin perderse.

Escalabilidad / portabilidad. NestJS es Node.js estándar; corre igual en Hostinger, en un VPS, en Render/Railway/Fly, o en un contenedor. No hay amarre. Y su modularidad es justamente lo que paga dividendos cuando Coco pase de script personal a producto con equipo: es una arquitectura que *ya* está lista para crecer.

Alternativa considerada: Fastify/Express serían más livianos para un uso puramente personal, pero dado que Coco “podría venderse”, la estructura de NestJS es una inversión que se amortiza en la etapa 2 y que además ya dominas.

Veredicto: NestJS.

Movimiento al escalar: el mismo código NestJS se despliega en infra profesional; se añaden réplicas del proceso detrás de un balanceador cuando el tráfico lo exija.

5.3 ORM — Prisma

Prisma es el eje de la portabilidad y de la mantenibilidad, y por eso merece capítulo propio. Da un cliente type-safe (autocompletado y verificación en compilación de cada consulta), migraciones versionadas (`schema.prisma` como fuente de verdad), y —crucialmente— **abstrae el motor de base de datos**. Es lo que permite decir “MariaDB hoy, gestionado mañana” con confianza.

Adecuación + mantenibilidad. Modelar el esquema, evolucionarlo con migraciones y consultarlo con tipos seguros reduce drásticamente los bugs de datos, que en finanzas son los más caros. Es, además, el ORM que ya usas en Vettx.

Matiz técnico (documentado): en desarrollo se usa `prisma migrate dev` contra una **MariaDB local** (necesita una *shadow database* que el hosting compartido no permite crear); a Hostinger solo se **aplican** migraciones con `prisma migrate deploy`, que no requiere shadow database. Esto define el flujo de trabajo de base de datos (ver sección 6).

Veredicto: Prisma, sin reservas. Es la pieza que hace barata la etapa 2.

5.4 Frontend — React + Vite + TypeScript (SPA)

Adecuación. Una SPA cubre bien una app de finanzas rica en pantallas y estado (dashboard, listados, formularios rápidos, revisión de importación). Vite da un desarrollo veloz y un bundle optimizado.

Mantenibilidad + talento. React es el framework de UI más difundido: máxima documentación, ecosistema y —relevante para la etapa 2— la mayor oferta de desarrolladores si algún día contratas. TypeScript de punta a punta (mismo lenguaje que el backend) permite **compartir tipos** y reduce errores de contrato.

Escalabilidad / portabilidad. Un frontend estático se sirve desde cualquier lado: la carpeta de Hostinger hoy, o un CDN gratuito (Cloudflare Pages, Netlify, Vercel) mañana — sin cambiar el código.

Nota para la etapa 2: si el producto necesita una web de marketing con SEO, eso se resuelve con un sitio aparte o un Next.js dedicado para la landing; **la app en sí** como SPA sigue siendo la decisión correcta. **Veredicto: React + Vite.**

5.5 Identidad — Firebase Authentication

Adecuación + seguridad. Delegar la autenticación a Firebase evita construir (y custodiar) el componente más delicado: hashes de contraseñas, 2FA, login con Google, verificación de correo, recuperación. El backend NestJS solo **verifica el token** con `firebase-admin`. Menos superficie de ataque, menos responsabilidad de seguridad sobre tus hombros.

Escalabilidad. Firebase Auth escala a millones de usuarios y su free tier es amplio: cubre de sobra la etapa 1 y buena parte de un producto temprano. En la etapa 2 sigue siendo una opción de producción legítima; si algún día se quisiera cambiar (a Auth0, Cognito, Clerk, o auth propia), los usuarios de Firebase son **exportables**, así que tampoco amarra de forma irreversible.

Veredicto: Firebase Auth. Es la decisión que más seguridad compra por menos esfuerzo, en ambas etapas.

5.6 Procesamiento de documentos — OCR en el cliente

Adecuación + costo. Correr Tesseract.js (imágenes) y pdf.js (PDF) en el navegador cumple tres objetivos de una vez: **privacidad** (los extractos nunca salen del equipo), **costo cero** (no se paga OCR en la nube ni CPU de servidor) y **escalabilidad implícita** (cada usuario aporta su propia CPU). Para Coco es una elección de diseño excelente.

El ajuste importante (mantenibilidad/escalabilidad): aunque hoy el OCR es 100% cliente, conviene implementarlo **detrás de una interfaz intercambiable** (un contrato `IOcrProvider` que hoy resuelve `TesseractProvider` en el navegador). ¿Por qué? Porque en la etapa 2, un producto vendible quizá quiera ofrecer OCR de servidor de mayor calidad o consistencia (por ejemplo, un microservicio propio con Tesseract/PaddleOCR en un VPS, o un OCR de pago como *tier*

premium). Si el pipeline de importación llama a una interfaz y no directamente a Tesseract, **añadir esa opción es enchufar una implementación nueva, no reescribir el módulo**. Esta es la única recomendación de diseño *nueva* que sale de este análisis.

Veredicto: OCR en el cliente, tras una interfaz `OcrProvider`.

6. Mantenibilidad (visión transversal)

Más allá de cada capa, el stack en conjunto es **fácil de mantener por una sola persona**, que es el criterio que más importa en la etapa 1:

- **Un solo lenguaje (TypeScript) de extremo a extremo.** Frontend y backend comparten lenguaje y tipos; menos context-switching, menos errores de contrato.
- **Fuente de verdad única para los datos.** `schema.prisma` + migraciones versionadas: el esquema evoluciona de forma segura y auditable, sin SQL suelto.
- **Estructura predecible.** NestJS impone un orden (módulo → controller → service → repository) que hace el código navegable.
- **Coincide con tu experticia.** NestJS + Prisma + TypeScript es el stack de Vettx: no aprendes nada nuevo para mantener Coco.
- **Seguridad delegada.** Firebase carga con lo más delicado de la autenticación.
- **Pruebas donde importan.** La lógica de dinero (prorrrateo, saldos, amortización, dedupe) se cubre con Jest; el resto se apoya en los tipos.

El flujo de trabajo de base de datos, derivado del análisis: **desarrollas contra una MariaDB local** (Docker, misma versión que Hostinger) donde `prisma migrate dev` funciona con su shadow database; **despliegas a Hostinger** aplicando migraciones con `prisma migrate deploy` a través de un **túnel SSH**. Mismo esquema en ambos lados, sin exponer la base a internet.

7. Escalabilidad y el plan de migración a la etapa 2

Aquí está la prueba de que la arquitectura “portable-first” cumple su promesa. Si Coco se vuelve vendible, la migración a infra profesional es por fases y **sin reescritura**:

7.1 Fotografía por fases

Pieza	Etapla 1 (personal, ≤50 usuarios)	Etapla 2 (producto)	¿Qué cambia en el código?
Frontend SPA	Carpeta del subdominio en Hostinger	CDN (Cloudflare Pages / Netlify / Vercel)	Nada (solo dónde se sube)
API NestJS	Node.js Web App de Hostinger	VPS / Render / Railway / Fly, con réplicas	Nada (variables de entorno)
Base de datos	MariaDB de Hostinger	MySQL/Postgres gestionado (réplicas, backups)	<code>DATABASE_URL</code> + migración de datos
Auth	Firebase (free tier)	Firebase (plan pago) o Auth0/Cognito	Nada o cambio de proveedor de token
OCR	Cliente (Tesseract.js)	Cliente + opción de servidor (microservicio)	Se enchufa un <code>OcrProvider</code> nuevo
Trabajos pesados	Cron ligero	Cola (BullMQ + Redis) si hace falta	Se añade, no se reescribe

7.2 Multi-tenancy: ya está resuelto

El obstáculo más común al convertir una app personal en SaaS es que “todo asume un solo usuario”. Coco **no** tiene ese problema: el modelo de datos scopea **cada fila por `user_id`** desde el inicio, y la API deriva ese `user_id` del token verificado (nunca del cliente). Pasar a multiusuario es **poblar más usuarios**, no rediseñar. Si el producto necesitara *organizaciones/equipos* (varias personas compartiendo finanzas), se añade una capa de `tenant_id/roles` sobre la base existente — evolución incremental, no reconstrucción.

7.3 Disparadores de migración (cuándo mover cada pieza)

No se migra por gusto, sino por señal:

- **CPU burstable throttling** o RAM al límite de forma sostenida → mover la API a VPS/PaaS.
- **Límite de conexiones de MariaDB** apretando con usuarios concurrentes → base gestionada con pooling.
- **Necesidad de SLA / uptime** (clientes pagando) → réplicas, backups gestionados, observabilidad.
- **Necesidad de OCR consistente de alta calidad** como feature vendible → `OcrProvider` de servidor.

8. Riesgos y techos honestos del “cero costo Hostinger”

Para que el análisis sea honesto, estos son los límites reales de la elección de etapa 1 (ninguno bloquea los 50 usuarios; todos son señales de la etapa 2):

Riesgo / techo	Impacto	Mitigación
CPU burstable con cuota horaria no publicada	Throttling si el uso se vuelve pesado y sostenido	Carga liviana (OCR en cliente) lo evita; migrar si aparece
RAM 1,5 GB (Business), un solo proceso	Sin autoescalado; caída = downtime hasta reinicio	Cloud Startup (3 GB, CPU dedicada) da margen; migrar en etapa 2
<code>max_user_connections</code> de MariaDB compartida	Cuello si hay muchas conexiones concurrentes	<code>connection_limit</code> bajo en Prisma; base gestionada en etapa 2
Respaldos manuales (cron + mysqldump)	Riesgo de pérdida de datos si falla el cron	Verificar restauración periódicamente; backups gestionados en etapa 2
Quirks del Node.js Web App (arranque, PATH de Node en SSH)	Fricción operativa ocasional	Documentado en el PRD (cap. 25)
Límites del free tier de Firebase	Aparecen solo a gran escala	Amplios para etapa 1; plan pago en etapa 2

La lectura correcta de esta tabla: **el costo cero de Hostinger es la decisión correcta para la etapa 1, y sus límites son exactamente las señales que indicarán cuándo pasar a la etapa 2** — no defectos que haya que arreglar hoy.

9. Reglas de diseño para no cerrar la puerta (checklist)

Para que la promesa “portable-first” se cumpla, el código de la etapa 1 debe respetar unas pocas reglas. La mayoría **ya están en el PRD**; se consolidan aquí:

- ☐ **Esquema en SQL portable.** Tipos estándar (`DECIMAL`, `ENUM`, `DATE`, `JSON`), sin funciones exclusivas de MariaDB. (Ya es el caso.)
- ☐ **Todo scopeado por `user_id`** en cada consulta Prisma, derivado del token. (Ya es el caso.)
- ☐ **API sin estado** (stateless): cada petición trae su token; nada de sesión en memoria del proceso. Habilita réplicas. (Ya es el caso.)
- ☐ **Configuración por entorno** (`@nestjs/config`, `.env` fuera de git): mover de Hostinger a otra infra es cambiar variables, no código. (Ya es el caso.)
- ☐ **OCR detrás de una interfaz `OcrProvider`**. (Ajuste nuevo recomendado por este análisis.)
- ☐ **Conector Prisma estándar** (no el driver adapter de MariaDB) y `connection_limit` bajo. (Ajuste documentado.)
- ☐ **Migraciones con Prisma** (dev en local, deploy en Hostinger): historial versionado que viaja a cualquier motor. (Ya es el caso.)
- ☐ **Frontend 100% estático** sin dependencias del servidor que lo sirve: portable a cualquier CDN. (Ya es el caso.)

10. Tabla-resumen de decisiones

Capa	Decisión (etapa 1)	Por qué	Movimiento en etapa 2
Frontend	React 19 + Vite + TS (SPA)	Estándar, mantenible, talento abundante, portable	Servir desde CDN gratuito
Backend	Node.js + NestJS + TS	Estructura para 10 módulos, = stack Vettx, portable	Réplicas en VPS/PaaS
ORM	Prisma	Type-safe, migraciones, abstrae el motor	Cambiar DATABASE_URL
Base de datos	MariaDB (Hostinger)	Gratis, madura, compatible con Prisma vía <code>mysql</code>	Gestionada (MySQL/Postgres)
Auth	Firebase Auth	Seguridad delegada, escala a millones, free tier	Plan pago o migrar proveedor
OCR	Cliente (Tesseract.js/pdf.js) tras OcrProvider	Privacidad + costo cero + escala con el cliente	Añadir opción de servidor
Despliegue	Node.js Web App + estático (Hostinger)	Cero costo, ya pagado	Lift-and-shift a infra pro

11. Veredicto final

El stack definido para Coco App —**React/Vite + NestJS + Prisma + MariaDB (Hostinger) + Firebase + OCR en el cliente**— es **la elección correcta** para el escenario “personal hasta ~50 usuarios hoy, vendible mañana”. No porque sea el más moderno o el más potente en abstracto, sino porque **maximiza el valor bajo tus restricciones reales**: corre gratis en la infraestructura que ya tienes, lo mantienes tú solo con el stack que ya dominas, aguanta 50 usuarios con holgura (gracias al OCR en el cliente), y —lo más importante— **no te amarra**: el día que Coco valga la pena venderse, se traslada a infraestructura profesional pieza por pieza, sin reescribir la lógica de negocio.

Sobre las dudas puntuales: **MariaDB no es un problema, es la opción correcta para esta etapa**; Claude Code tenía razón en el nombre del motor, y el PRD ya quedó corregido. Y la escalabilidad no exige cambiar el stack hoy: exige **respetar las reglas de diseño portables** del capítulo 9 — que en su mayoría ya están en el PRD — y añadir un único ajuste: **tratar el OCR como una pieza intercambiable**.

En una frase: **construye simple y portable para 50 usuarios a costo cero; el mismo código estará listo para escalar cuando toque**.